VERIQTA

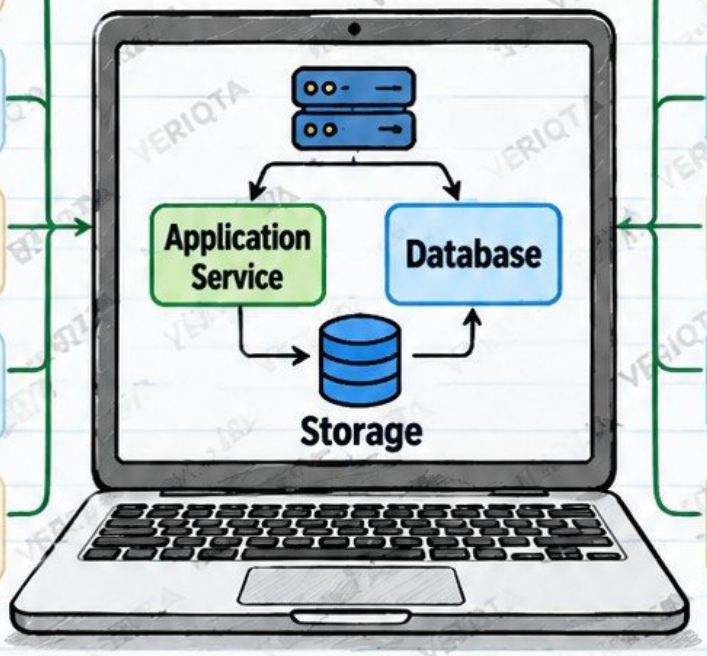
VERIQTA

System Design for Beginners:

10 Systems You'll Be Asked About

Learn how real applications are designed, scaled and kept reliable.
With clear diagrams, real-world scenarios and practical engineering insights.

- URL Shortener
- Chat Application
- Video Streaming
- E-commerce
- Ride-Hailing



- Social Media Feed
- Notification System
- File Storage
- Payment Processing
- High-Traffic Web Application

REAL-WORLD ARCHITECTURES | **SCALING STRATEGIES** | **RELIABILITY AND FAILURE HANDLING** | **PRACTICAL ENGINEERING INSIGHTS**

20 PAGES OF PRACTICAL, BEGINNER-FRIENDLY NOTES

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Design a URL Shortener Like Bitly

Scenario: A company needs to turn long URLs into short links that redirect users reliably.

1 How a short link maps to an original URL

- A short URL (e.g. https://v.q/abc123) is a unique code that maps to the full original URL.
- The code (abc123) is a key in a database that stores the destination URL, creation time and optional expiration date.
- When a user visits the short link, the system looks up the code and redirects them to the original URL.

ID	Short Code	Original URL	Expires At
1	abc123	https://example.com /blog/post	2026-12-31
2	xy9kL2	https://veriqta.com	2027-01-15
3	mN7pQ8	https://youtube.com /watch?v=...	NULL

2 How a user request travels through the system



- **DNS:** Resolves v.q to an IP address
- **Load Balancer:** Distributes requests to application servers
- **URL Service (Application):** Looks up the short code and returns a redirect response
- **Database:** Stores short code, original URL and expiration date

3 How unique short codes are generated

- Use a random or sequential ID (e.g. 123456789).
- Convert the ID to a short, non-sequential string using Base62 (A–Z, a–z, 0–9) to keep it short.
- Check if the code already exists to avoid collisions (rare with large ID space).
- Example: 123456789 → 8mF2kL (Base62).

4 Why HTTP redirects use 301 or 302

- 301 Moved Permanently
  - Used when the link will not change.
  - Browsers and search engines can cache it.
- 302 Found (Temporary Redirect)
  - Used when the link might change or expire.
  - Tells the browser to fetch the latest destination.
- Both return the original URL in the Location header.

5 Where engineers store link mappings

- A database (e.g. PostgreSQL, MySQL) stores:
  - Short code (unique)
  - Original URL
  - Created at (timestamp)
  - Expires at (optional)
  - Clicks count (optional)
  - User ID (if links are per user)

Example table structure

```
id            BIGINT
short_code    VARCHAR(10)  UNIQUE
original_url  TEXT
Created_at    TIMESTAMP
expires_at    TIMESTAMP NULL
clicks        BIGINT DEFAULT 0
```

Junior Engineer Tip

- Short codes should be unpredictable to prevent guessing and abuse. Use a large ID space (e.g. Base62) and rate limiting to prevent scraping or brute-force attacks.

Common Mistakes

- Using sequential short codes (easy to guess).
- Forgetting to handle expired links.
- Returning 200 OK instead of a redirect (won't work).
- Not checking for code collisions.

Practice This

- Design a simple table for storing short links.
- Write a function that converts a number to a Base62 string.
- Decide when to use 301 vs 302 for your use case.
- What additional fields would you store for a production system?

VERIQTA

Scale a URL Shortener to Millions of Clicks

Scenario: A popular short link suddenly receives thousands of requests per second.

1 Why repeated database lookups become expensive

- Each redirect requires reading the short code from the database.
- As traffic grows, the database becomes a bottleneck (CPU, connections, I/O).
- Repeated lookups for popular links waste resources.
- High latency increases response time for every user.

Without Caching (High Load)

- Many repeated DB queries
- Higher latency
- Database CPU and I/O increase
- Can cause timeouts or failures

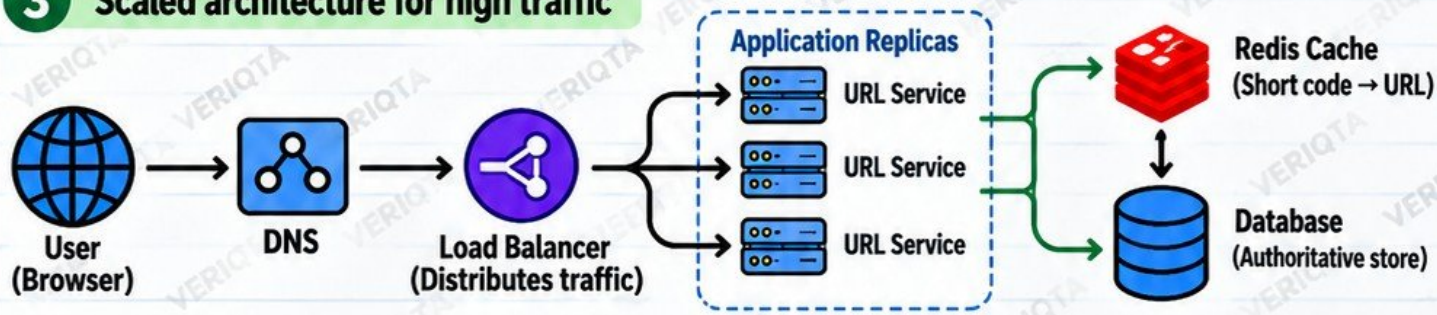
2 How Redis caching speeds up redirects

- Store short code → original URL in Redis (in-memory cache).
- Redirects are served from cache (very fast, milliseconds).
- Only query the database if the code is not in cache (cache miss).
- Set a TTL (time to live) so outdated links expires automatically.

With Redis Cache (Low Latency)

- Most requests served from cache
- Faster redirects
- Lower database load
- Handles high traffic efficiently

3 Scaled architecture for high traffic



4 What happens when a short code is missing or expired

Situation	HTTP Response	Explanation
Valid short code	301 or 302	Redirects to original URL.
Short code not found	404 Not Found	Code does not exist.
Expired short code	410 Gone	Code existed but has expired.

Why return 410 for expired?

- 410 Gone clearly tells the client that the link is no longer valid.
- Better for user experience and SEO than 404.

5 How to investigate slow redirects

1. Check application latency and error rate.
2. Check Redis cache hit rate vs. cache misses.
3. Verify database CPU, connections and slow queries.
4. Look at load balancer and application replica health.
5. Check if a specific short code is causing high traffic.
6. Review recent deployments or configuration changes.

6 Engineering decision: When to cache a redirect?

Consideration	Best Practice
Cache duration	Set a TTL (e.g. 1 hour or 1 day) based on how often links change.
Prevent stale destinations	Use short TTL, or actively invalidate cache when a link is updated or deleted.
Popular links	Cache aggressively (longer TTL).
Sensitive or frequently changing links	Use shorter TTL or skip caching.

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

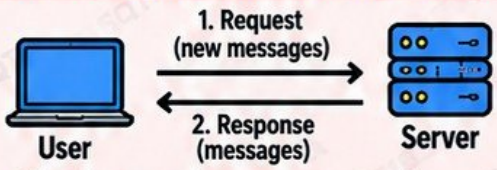
Design a Real-Time Chat System Like WhatsApp

Scenario: Two users must exchange messages even when they are connected to different application servers.

1 Why ordinary HTTP requests are not enough

- HTTP is **stateless**. The client sends a request and the server responds, then the connection closes.
- To get new messages, the client must keep polling, which wastes bandwidth and increases delay.
- Polling can take seconds, so it is not efficient for real-time chat.
- Real-time chat needs a **persistent**, two-way connection.

HTTP Polling (Not Efficient for Real-Time)



- Client keeps sending requests repeatedly.
- Higher latency and unnecessary network traffic.
- Not suitable for instant messaging.

2 How WebSocket connections enable two-way communication

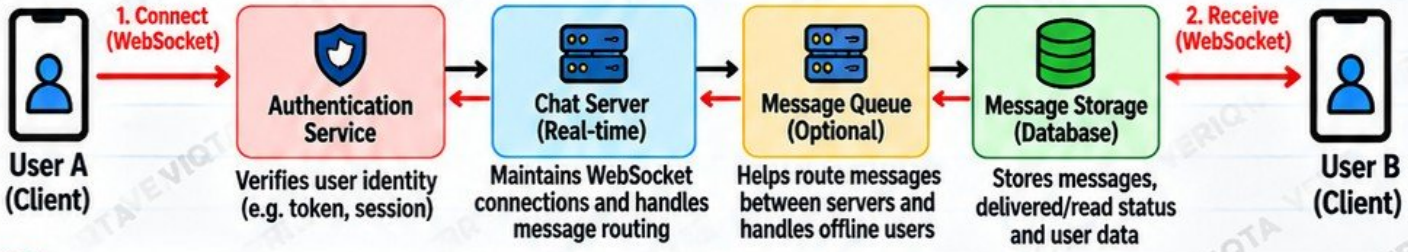
- WebSocket keeps a persistent connection open between client and server.
- Both the client and server can send messages at any time.
- Once connected, messages are pushed instantly without constant polling.
- This provides low latency and real-time delivery.

WebSocket (Two-Way, Persistent Connection)



- Open once, keep it open.
- Low latency and real-time communication.
- Used by WhatsApp, chat apps and many real-time systems.

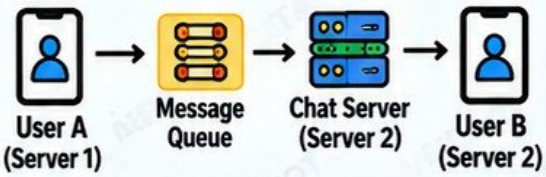
3 How the chat system components work together



4 How messages move between users on different servers

1. User A sends a message thrugh an open WebSocket connection.
2. The chat server validates the user and accepts the message.
3. The message is stored in the database (for history and offline delivery).
4. The server checks if User B is online and connected to this server.
5. If User B is connected to a different server, the message is routed using a message queue or internal service.
6. The server pushes the message to User B over their WebSocket connection.

Message Flow (Different Servers)



5 Delivered and read acknowledgments

Status	What it means	How it works
Sent	Message successfully sent from your device to the server.	Client receives confirmation from the server.
Delivered	Message delivered to the recipient's device.	Recipient's app receives the message.
Read	Recipient has opened and seen the message.	Recipient's app sends a read receipt back to the server.

Important Notes

- If the recipient is offline, the message is stored and delivered when they reconnect.
- Read receipts are optional and controlled by user privacy settings.
- Multiple devices (phone, web, tablet) can be connected at the same time.
- The system must handle network failures, retries and duplicate messages.

VERIQTA

VERIQTA

What Happens When Chat Users Go Offline?

Scenario: A user loses internet access while someone sends them a message.

1 Messages are durably stored before delivery

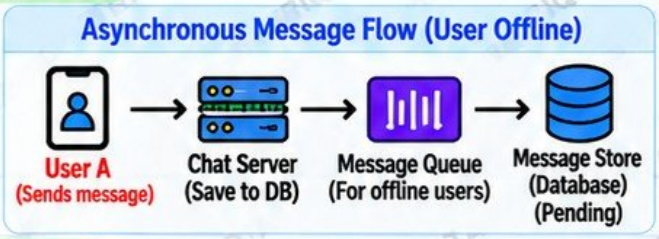
- When a message is sent, it is immediately written to a durable message store (database) before it is delivered to the recipient.
- This ensures the message is not lost, even if the recipient is offline.
- The message is marked as "pending" until it is delivered.

Why durable storage is important

- Users can be offline for minutes, hours or days.
- Servers can fail or restart.
- Messages must still be available for delivery later.

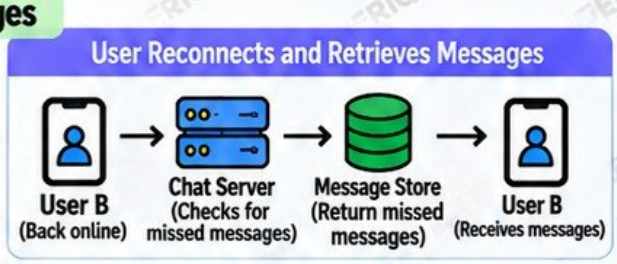
2 How message queues support asynchronous delivery

- When the recipient is offline, the message is placed in a message queue.
- A background worker keeps trying to deliver the message until the user is online.
- Queues help handle spikes, retries and temporary failures without losing messages.



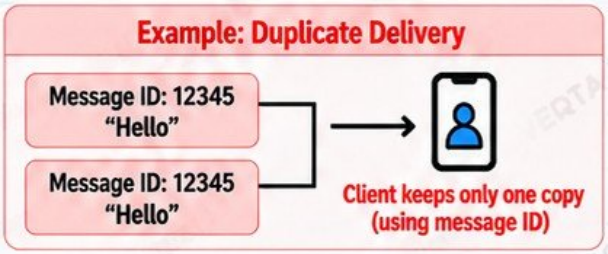
3 How reconnecting clients retrieve missed messages

- When the user comes back online, the client authenticates.
- The server checks for undelivered messages (using the user's last seen message ID or timestamp).
- All missed messages are sent to the client in order.
- Messages are then marked as delivered (and read when opened).



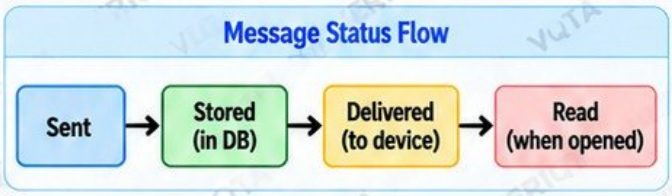
4 Why retries can create duplicate deliveries

- If the server does not receive a confirmation, it retries delivery.
- The original delivery might have actually succeeded, but the confirmation was lost due to a network issue.
- This can cause the same message to be delivered more than once.
- To prevent this, each message has a unique ID, and the client discards duplicates.



5 Message IDs, ordering and deduplication

Concept	Purpose
Message ID (unique)	Identifies each message (e.g. UUID).
Ordering	Messages are stored with a timestamp or sequence number to maintain order.
Deduplication	Client ignores duplicate message IDs.
Acknowledgments	Client sends delivered and read receipts using the message ID.



6 Production failure: Server restarts during delivery

What can happen?

- Messages already stored in the database are not lost.
- Some messages may be delivered more than once if the server retries after restart.
- Messages in memory (not yet persisted) can be lost.
- Some messages may be delayed until the server recovers and workers resume.

How to reduce these issues

- Always persist messages before sending.
- Use durable queues (e.g. Kafka, RabbitMQ, SQS).
- Use message IDs and deduplication on the client.
- Use acknowledgments and retries with backoff.
- Monitor queue depth, worker health and delivery failures.
- Design for at-least-once delivery, and handle duplicates.

Junior Engineer Tip

- Never rely only on in-memory storage for messages.
- Always design for retries and duplicate deliveries.
- Test failure scenarios like server restarts, network loss and queue delays.

VERIQTA

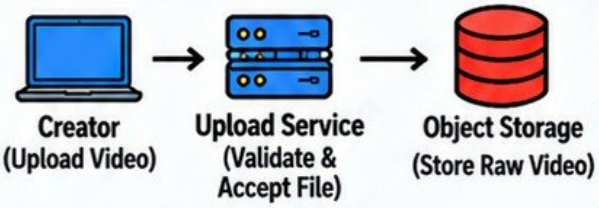
Design a Video Streaming Platform Like YouTube

Scenario: A creator uploads a video that must be available to viewers using different devices and network speeds.

1 How a video enters an upload service

- The creator uploads a video using a web or mobile app.
- The client sends the file to an upload service (usually using a resumable or multipart upload for large files).
- The upload service validates the file (size, type, malware scan).
- The raw video is stored in object storage (e.g. Amazon S3, Google Cloud Storage) with a unique file ID.

Upload Flow



2 Why object storage holds large media files

- Videos are large files and not suitable for traditional databases.
- Object storage is designed for large, durable and cost-efficient storage.
- It automatically replicates data for high availability.
- The application stores only the file URL or object key, not the actual video inside the database.

Key Benefits of Object Storage

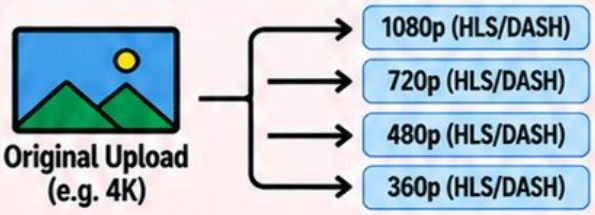


- Handles very large files (GBs to TBs).
- Highly durable with replication.
- Cost-effective compared to traditional storage.
- Accessible from multiple services.
- Ideal for video, images and backups.

3 Why videos are transcoded into different resolutions

- A single video is converted into multiple resolutions (e.g. 1080p, 720p, 480p, 360p).
- Different devices and network speeds require different video quality.
- Smaller resolutions reduce buffering on slow networks.
- Adaptive bitrate streaming allows the player to switch quality based on the user's internet speed.

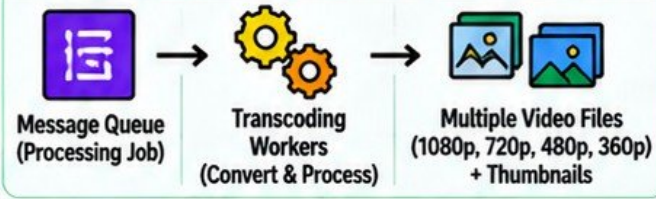
Example Transcoded Versions



4 How background workers process uploaded files

- A message is placed in a queue after the upload completes.
- Background workers pick up the job and transcode the video into different resolutions.
- Workers also generate thumbnails, extract metadata, and create video segments (e.g. HLS or DASH).
- When processing is complete, the database is updated with the available video versions and metadata.

Video Processing Pipeline (Background Workers)



5 Metadata and video files are stored separately

- Video files are stored in object storage.
- Metadata (title, description, duration, available resolutions, thumbnail URL, owner) is stored in a database (e.g. PostgreSQL, MySQL, or a NoSQL database).
- This separation allows fast metadata queries and scalable file storage.

Storage Design

Metadata (Database)	
video_id	12345
title	My Travel Vlog
description	A day in Paris
duration	600 seconds
owner_id	user_123
thumbnail_url	s3://.../thumb.jpg
status	ready

Video Files (Object Storage)



12345_1080p.mp4
12345_720p.mp4
12345_480p.mp4
12345_360p.mp4
12345_thumb.jpg

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

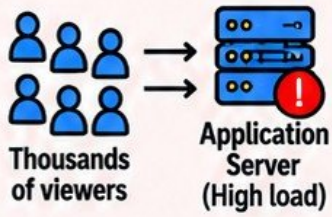
How Netflix and YouTube Deliver Videos at Scale

Scenario: Thousands of viewers begin watching the same video.

1 Why streaming directly from one application server is inefficient

- A single application server cannot handle thousands or millions of viewers.
- Video files are large and require high bandwidth.
- Serving all traffic from one server increases latency and can cause the server to fail.
- It wastes resources and does not scale globally.
- Instead, videos are stored in object storage and delivered through a Content Delivery Network (CDN).

Direct from Application Server (Not Scalable)

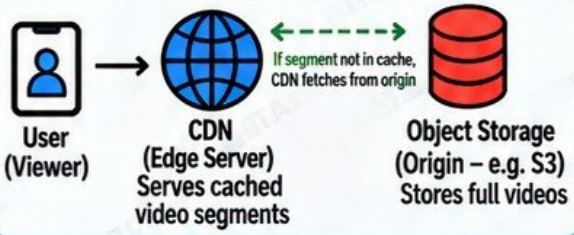


- High bandwidth usage
- High latency
- Server overload
- Single point of failure
- Not scalable globally

2 How a Content Delivery Network (CDN) serves video segments

- Videos are stored in object storage and split into small segments (e.g. 2–10 seconds).
- These segments are cached on CDN servers around the world (edge locations).
- When a user requests a video, they are served from the nearest CDN server.
- This reduces latency, saves bandwidth and allows millions of viewers to watch the same video efficiently.

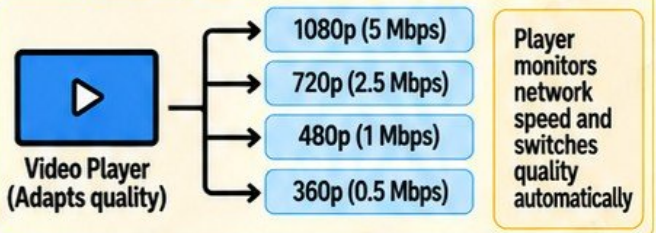
Video Delivery with CDN



3 How adaptive bitrate streaming adjusts video quality

- Videos are encoded into multiple resolutions/bitrates (e.g. 1080p, 720p, 480p, 360p).
- The player (HLS or DASH) automatically switches between qualities based on the user's network speed.
- If the connection is slow, the player switches to a lower quality to avoid constant buffering.
- If the connection improves, it switches to a higher quality.

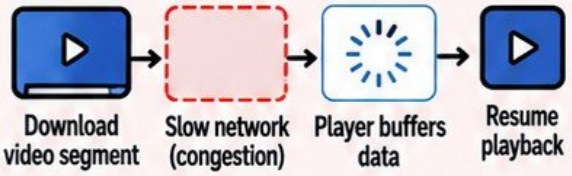
Adaptive Bitrate Streaming (HLS / DASH)



4 Why buffering happens during network congestion

- Buffering occurs when the video player cannot download segments fast enough.
- This can be caused by slow user internet, network congestion, high latency or CDN issues.
- The player pauses and buffers data to prevent playback interruptions.
- Adaptive bitrate helps, but severe congestion can still cause buffering even on lower quality.

Buffering During Network Congestion



5 How cache hit rates and startup latency reveal problems

Metric	What it means	Why it matters
Cache hit rate	Percentage of requests served from CDN cache	Low hit rate increases load on origin storage and causes higher latency.
Playback startup latency	Time from play button to video starting	High startup latency means CDN, network or origin issues.
Rebuffering rate	How often the video stops to buffer	High rate indicates network or CDN performance problems.
Segment download time	Time to download each video segment	Slow downloads can cause poor playback quality.

6 Troubleshooting: Thumbnails load but playback keeps buffering

Components to investigate

1. CDN (edge server performance and cache hit rate).
2. Origin storage (is it slow or failing?).
3. Network (latency, packet loss, ISP issues).
4. Video segments (are all qualities available?).
5. Transcoding output (corrupt or missing segments).
6. Player configuration (HLS/DASH settings).
7. Monitoring and logs (CDN metrics, origin metrics, errors).

VERIQTA

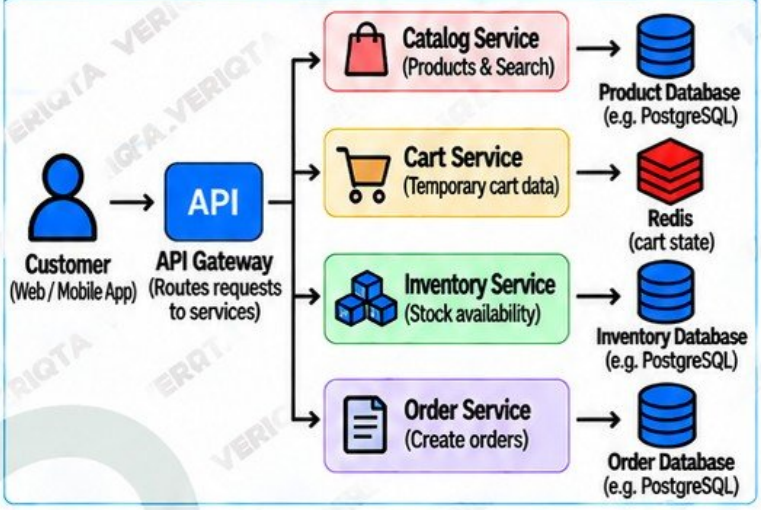
Design an Online Store Like Amazon

Scenario: A customer searches for a product, adds it to a cart, and proceeds to checkout.

1 How the main services interact

- The customer uses the web or mobile app to search for products.
- The API gateway routes requests to the appropriate services.
- The catalog service provides product information and search results.
- The cart service stores the user's cart items (usually in Redis).
- The inventory service tracks stock availability.
- At checkout, the order service creates the order and reserves inventory.

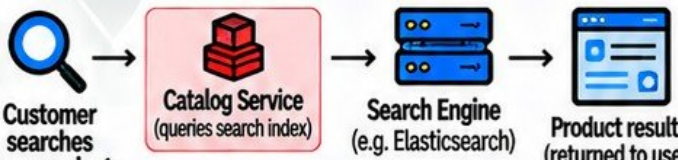
High-Level Architecture



2 Product catalog and search

- Products are stored in a product database (e.g. PostgreSQL, MySQL).
- A search engine (e.g. Elasticsearch, OpenSearch) indexes product details for fast search.
- The catalog service returns product information, images, prices, and availability.

Product Search Flow



3 Shopping cart vs inventory (different purposes)

Shopping Cart (Temporary and User-Specific)



- Stores items a user wants to buy.
- Usually kept in Redis for fast access.
- Can expire if the user is inactive.
- Does not reduce actual inventory.

Inventory Record (Actual Stock)



- Tracks the real stock available in the warehouse.
- Updated when an order is placed (not when item is added to cart).
- Prevents overselling.
- Stored in a dedicated inventory database.

4 How databases store key data

Products Database



- Product ID
- Name, description
- Category
- Price
- Images
- Attributes (size, color, etc.)

Orders Database



- Order ID
- Customer ID
- Product items
- Total amount
- Payment status
- Order status (pending, shipped, etc.)

Customers Database



- Customer ID
- Name, email
- Shipping address
- Billing address
- Order history
- Account details

5 Why Redis can support cart state



- Redis is fast and ideal for temporary cart data.
- Cart data has a short lifespan.
- It reduces load on the main database.
- It supports quick add, remove, and update operations.
- Cart data can be stored with an expiration time (TTL).

6 Why service boundaries matter

- Separating catalog, cart, inventory, and order services makes the system easier to scale.
- Each service can have its own database and technology.
- Changes to one service do not affect others.
- It improves reliability, maintainability, and team productivity.
- As the store grows, each service can be scaled independently.

VERIQTA

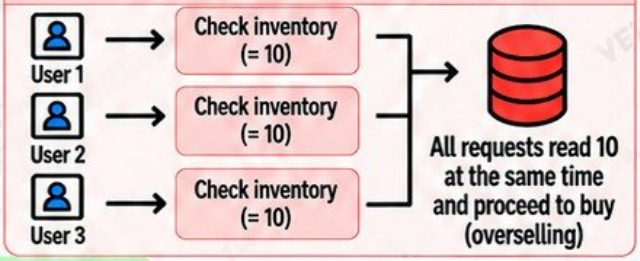
Prevent Overselling During a Flash Sale

Scenario: Only 10 products remain, but 1,000 customers attempt to purchase them simultaneously.

1 How concurrent requests create race conditions

- Many customers send purchase requests at the same time.
- If multiple requests read the same inventory value (e.g. 10), they all think the item is available.
- Without proper coordination, all requests may succeed, leading to overselling.

Race Condition (Without Protection)



2 Why checking inventory and updating must be coordinated

- Checking if an item is available and reducing the quantity must be done as a single, coordinated operation.
- If these steps are separated, multiple requests can pass the check before the quantity is updated.
- Use atomic operations, database traansactions or locking to ensure only one request updates the inventory at a time.

Coordinated Update (With Lock or Transaction)



3 How atomic database updates prevent overselling

- Use a single atomic SQL statement with a condition (e.g. quantity > 0).
- The database ensures only one request can reduce the quantity at a time.
- If the quantity is already 0, the update fails and the order is rejected.

Atomic Update Example (SQL)

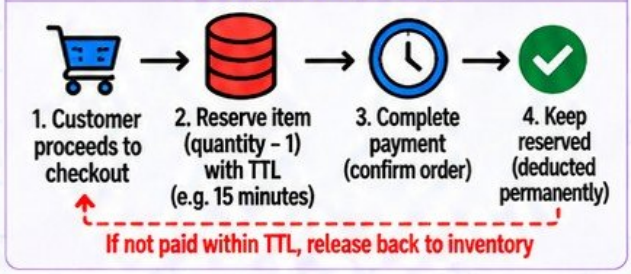
```
UPDATE products
SET quantity = quantity - 1
WHERE id = 123
AND quantity > 0;
```

- If the update succeeds (rows affected = 1), the item is reserved.
- If the update fails (rows affected = 0), item is out of stock.

4 How inventory reservations and expiration work

- When a customer proceeds to checkout, the system reserves the item for a short period (e.g. 10–15 minutes).
- The reserved quantity is not available for other customers.
- If the customer completes payment, the reservation is confirmed and the item is deducted permanently.
- If the reservation expires, the item is automatically released back to inventory.

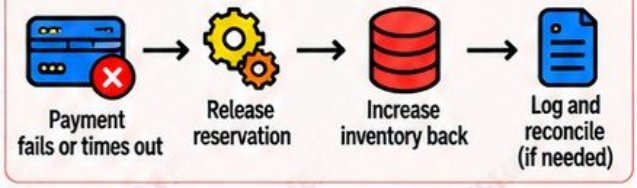
Inventory Reservation Flow



5 How payment failures trigger reservation release

- Payment can fail due to insufficient funds, network issues or provider errors.
- If payment fails or times out, the reservation must be released.
- A background job can automatically expire old reservations and return items to inventory.
- Reconciliation processes ensure inventory remains accurate even if some events are delayed.

Payment Failure Flow



6 Engineering decision: When to deduct inventory?

Deduct when added to cart	Deduct when checkout reserves
✗ High risk of items being locked by users who never checkout.	✓ More accurate and fair for all customers.
✗ Can lead to poor user experience.	✓ Items are reserved for a short time (e.g. 10–15 minutes).
✓ Generally not recommended for most online stores.	✓ Released automatically if payment fails or expires.
	✓ Used by most large e-commerce platforms.

Key Takeaways

- Use atomic updates or database transactions.
- Implement inventory reservations with TTL.
- Release items when payment fails or expires.
- Monitor inventory, failed payments and reservation queue health.
- Always reconcile inventory to handle edge cases.

VERIQTA

Design a Ride-Hailing System Like Uber

Scenario: A passenger requests a ride, and the platform must find an available driver nearby.

1 System overview

- Riders request a ride using a mobile app.
- The system finds nearby available drivers.
- A matching service selects the best driver.
- The driver accepts the ride and navigates to the pickup location.
- The trip status changes from requested to completed.
- Both rider and driver locations are updated in real time.

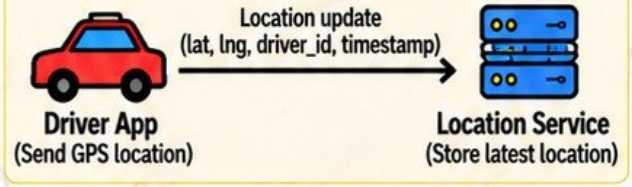
High-Level Architecture



2 How drivers send location updates

- The driver app sends GPS location updates at regular intervals (e.g. every 5–10 seconds).
- Updates are sent using a lightweight API or WebSocket connection.
- The system stores the latest location with a timestamp.
- Old locations are expired to keep the data accurate.

Driver Location Updates



3 How the system finds nearby drivers using geospatial indexing

- Driver locations are stored with geospatial indexes (e.g. Redis Geo, PostGIS, or Elasticsearch).
- The system queries for drivers within a certain radius (e.g. 2–5 km) of the passenger.
- This allows fast lookup of nearby available drivers.
- Only drivers who are online and not on a trip are returned.

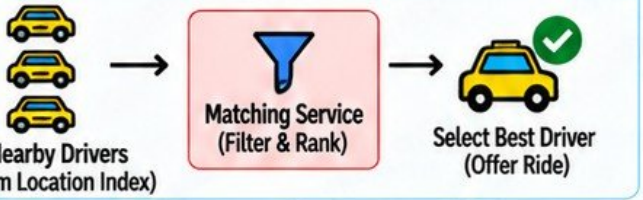
Geospatial Indexing Example (Redis Geo)



4 How the matching service selects eligible drivers

- The matching service filters drivers based on:
  - Proximity (nearest drivers first)
  - Driver availability (online and not on a trip)
  - Driver rating and acceptance rate
  - Estimated time of arrival (ETA)
  - Other factors (vehicle type, rider preferences)
- The best driver is offered the ride, and the system waits for acceptance.

Driver Matching Flow



5 Trip state changes



6 Why location freshness matters

Drivers move continuously, so old locations quickly become inaccurate.

Using stale locations can result in:
- Matching a driver who is far away
- Longer pickup times
- Poor user experience

The system must expire old location data (e.g. after 30–60 seconds) and keep only recent updates.

7 Example database design (simplified)

Table	Key Fields
drivers	id, name, status, rating, current_location, updated_at
riders	id, name, phone, rating
trips	id, rider_id, driver_id, pickup_location, dropoff_location, status, start_time, end_time, fare
locations (geospatial)	driver_id, lat, lng, timestamp (with geospatial index)

VERIQTA

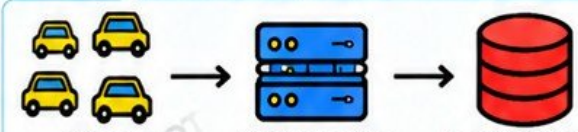
Handle Thousands of Ride Requests Simultaneously

Scenario: Heavy rain creates a sudden increase in ride requests across a city.

1 How high-frequency location updates affect system capacity

- During peak time, thousands of drivers send location updates every few seconds (e.g. every 3–5 seconds).
- This creates a very high write and network load on the system.
- The system must handle millions of location updates per minute.
- Efficient infrastructure (load balancers, scalable services, and geospatial databases like Redis Geo) is required.

High-Frequency Location Updates



2 Why location data may be slightly stale

- Location updates take time to travel over the network.
- Drivers may have weak signals or intermittent connectivity.
- The system may process updates asynchronously using queues.
- To reduce load, the app may send updates at intervals (not continuously).
- A location can be a few seconds old, but still accurate enough for matching nearby drivers.

Reasons for Slightly Stale Location Data



3 How queues and timeouts affect driver matching

- Ride requests and location updates are placed in queues (e.g. Kafka, RabbitMQ, SQS).
- Matching workers consume the requests and find available drivers.
- If the queue is slow or workers are overloaded, matching takes longer.
- Timeouts prevent requests from waiting forever.
- If no driver is found within the timeout, the request is retried or marked as failed.

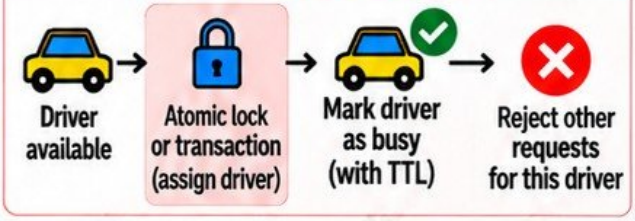
Ride Request Processing Flow (With Queues)



4 How to prevent one driver from accepting conflicting assignments

- Use atomic updates or distributed locks when assigning a driver.
- Mark the driver as "busy" immediately after assignment.
- Use a TTL (time to live) on the assignment in case of failures.
- Double-check driver availability before confirming.
- Reject new assignments if the driver already has an active trip.

Prevent Conflicting Assignments



5 How to monitor matching latency and request failures

Metric	What it means	Why it matters
Matching latency	Time from request to driver assigned	High latency indicates queue or worker issues.
Request success rate	Percentage of requests that get a driver	Low rate means capacity or supply issues.
Queue length	Number of pending ride requests	High length indicates workers cannot keep up.
Driver availability	Number of online and active drivers	Low availability increases wait time.
Location update rate	Number of location updates per second	Helps detect ingestion or network issues.

6 Production failure: Driver accepts trip, but passenger app still shows "Searching."

How would you investigate?

1. Check if the driver assignment was successfully written to the database.
2. Verify real-time event delivery (WebSocket / push notification) from the backend to the passenger app.
3. Check message queue delays or failed events.
4. Confirm if the passenger app is subscribed to the correct channel or rider ID.
5. Look for cache inconsistencies (e.g. stale "searching" state in Redis).
6. Check logs for errors in the matching service, notification service, and mobile app.
7. Verify network issues or client-side bugs in the passenger app.

VERIQTA

Design an Instagram-Style News Feed

Scenario: A user opens an application and expects to see recent posts from accounts they follow.

1 How posts, followers, and feed records are stored

- Posts are stored in a post store (e.g. database or object storage for media files).
- Follower relationships are stored in a social graph table (e.g. user_followers).
- Feed records (user timeline) are stored separately, often in a fast store like Redis or a NoSQL database.
- Each post includes metadata (post_id, user_id, caption, media_url, created_at, etc.).

Data Storage Model (Simplified)

Users & Followers (Social Graph)

user_id	123
username	alice
follows	[456, 789]

Posts (Post Store)

post_id	987
user_id	456
caption	My trip!
media_url	s3://image.jpg
created_at	2026-10-28

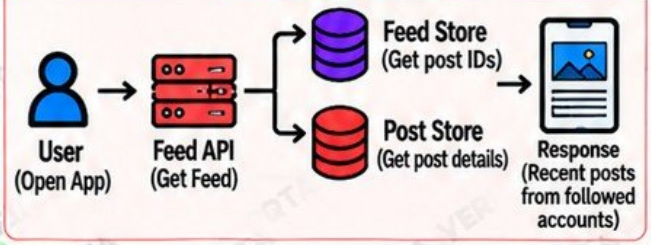
Feed Records (Feed Store)

user_id	123
post_ids	[987, 654, 321]
created_at	sorted by time

2 How the application retrieves posts from followed accounts

- The application gets a list of account IDs the user follows.
- It retrieves recent posts from those accounts (using fan-out on read or fan-out on write).
- Posts are ranked and sorted by timestamp or a relevance algorithm.
- Media files (images/videos) are fetched from a CDN for fast delivery.
- Only a limited number of posts are returned (pagination).

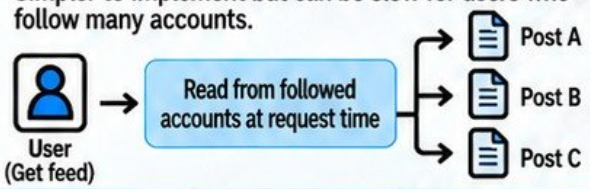
Feed Retrieval Flow



3 What fan-out on read and fan-out on write mean

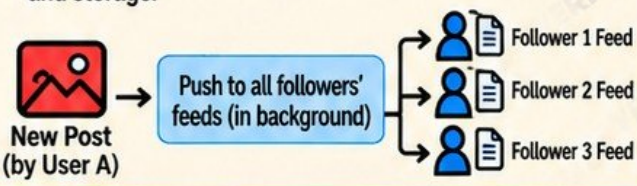
Fan-out on Read

- When a user opens the app, the system fetches recent posts from all followed accounts at read time.
- Good for users who follow a small number of accounts (e.g. < 500).
- Simpler to implement but can be slow for users who follow many accounts.



Fan-out on Write

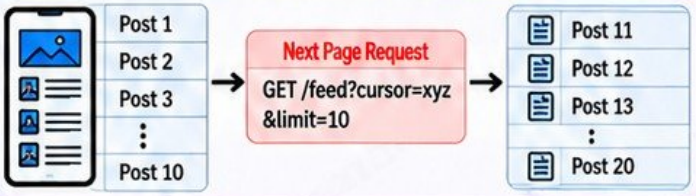
- When a user creates a post, the system pushes the post to the feeds of all their followers.
- Good for large, high-traffic systems.
- Faster feed retrieval but requires more write capacity and storage.



4 How pagination loads feed content

- The feed is returned in small portions (e.g. 10–20 posts).
- A cursor or timestamp is used to request the next set of posts.
- This reduces response size, improves performance, and saves bandwidth.
- The client requests the next page when the user scrolls.

Pagination Example



5 Why a feed is not simply one database query

- A user follows many accounts, so posts can come from different sources.
- Posts must be ranked, filtered (blocked accounts, privacy), and de-duplicated.
- Feed data is often spread across multiple services and stores.
- A single query would be slow, complex, and not scalable.

High-Level Feed Architecture



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

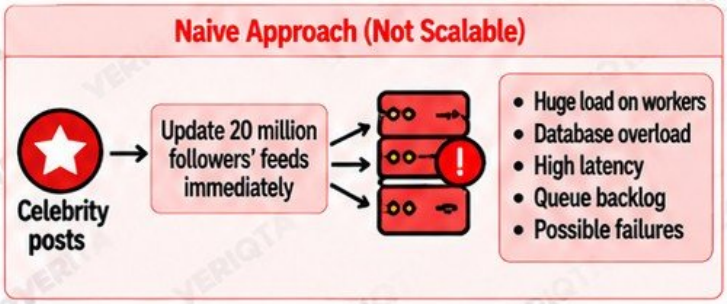
VERIQTA

What Happens When a Celebrity Posts?

Scenario: An account with 20 million followers publishes a new post.

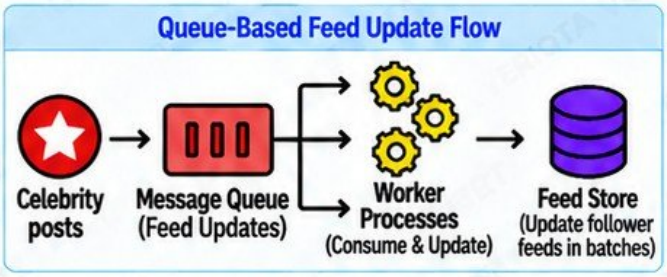
1 Why updating every follower's feed immediately can overwhelm workers

- A celebrity with 20 million followers means 20 million feed updates for a single post.
- If the system tries to update every follower's feed at once, it creates a massive spike in traffic.
- This can overwhelm background workers, databases, and caches.
- It may cause delayed updates, timeouts or system failures.



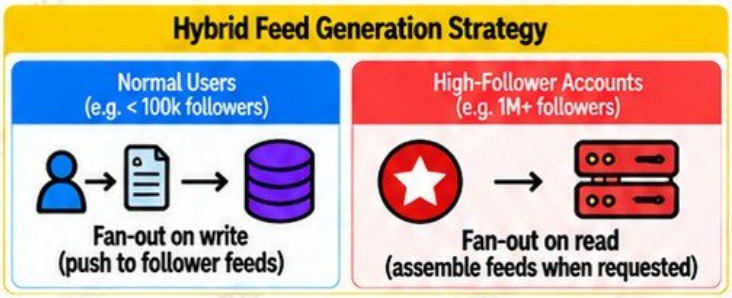
2 How background queues distribute feed updates

- The post is added to a message queue (e.g. Kafka, RabbitMQ, SQS).
- Multiple worker processes consume the queue in parallel.
- Each worker updates a batch of followers' feeds.
- The work is distributed over time to avoid overload.
- Retries and monitoring ensure all updates are completed.



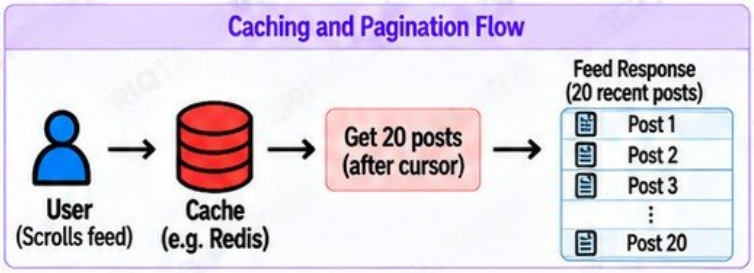
3 Why hybrid feed generation works for high-follower accounts

- For normal users (small to medium followers), use fan-out on write (push to follower feeds).
- For celebrities (millions of followers), use fan-out on read (assemble feeds when requested).
- This hybrid approach reduces write load while still showing posts to followers quickly.



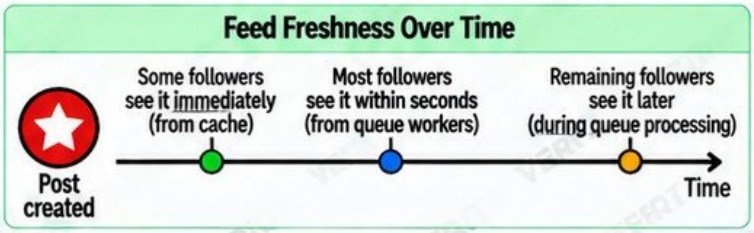
4 How caching and pagination reduce database pressure

- Feed results are cached in memory stores (e.g. Redis) for fast access.
- Pagination loads a small number of posts (e.g. 10–20) instead of all posts.
- This reduces database load, improves response time and saves bandwidth.



5 How delayed updates affect feed freshness

- Background queues may take a few seconds or minutes to update all followers' feeds.
- Some followers may see the post immediately, while others see it later.
- Feed freshness depends on queue load, worker capacity and system performance.
- This delay is usually acceptable and prevents system overload.



6 Engineering decision: When to generate feeds in advance, and when to assemble them when requested?

Approach	When to use	Advantages	Disadvantages
Generate feeds in advance (Fan-out on write)	Small to medium accounts (e.g. < 100k followers)	Fast read, simple design, consistent experience	High write load, not scalable for large accounts
Assemble feeds on request (Fan-out on read)	High-follower accounts (e.g. 1M+ followers)	Scales to millions of followers, less write load	Slightly higher read latency, more complex logic
Hybrid approach	Most production systems	Best balance of performance, scalability and cost	Requires more engineering effort and monitoring

Key Takeaways

- Do not update millions of followers at once.
- Use background queues and multiple workers.
- Use a hybrid approach (write for small accounts, read for large accounts).
- Cache feed results and use pagination.
- Accept slight delays in feed freshness to maintain system stability.

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

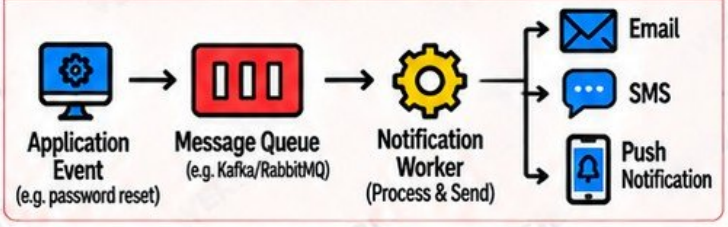
Design a Notification System

Scenario: An application must send password-reset emails, payment alerts, and mobile push notifications.

1 How an event triggers a notification

- A user action or system event (e.g. password reset, payment success, new message) creates a notification event.
- The application publishes an event to a message queue with the necessary details (user_id, event_type, data).
- The event is processed asynchronously so the main application remains fast and available.

Event to Notification Flow



2 How notification workers process queued jobs

- Multiple worker instances consume events from the queue.
- Each worker reads the event, checks user preferences, and prepares the message.
- Workers call the appropriate provider (email, SMS, or push) to send the notification.
- Failures are retried with exponential backoff and moved to a dead letter queue if needed.

Notification Worker Processing



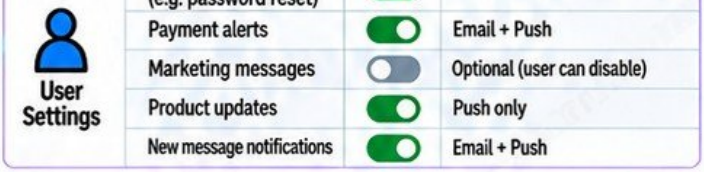
3 How email, SMS, and push providers differ

Channel	Common Providers	Best For	Characteristics
Email	SendGrid, Amazon SES, Mailgun	Account updates, receipts, password reset	• Cost effective • Slower delivery (seconds) • Supports rich content (HTML)
SMS	Twilio, AWS SNS, MessageBird	Critical alerts, 2FA codes	• Very high delivery rates • Short messages (limited characters) • Higher cost per message
Push	Firebase (FCM), Apple APNs, OneSignal	In-app activity, marketing, reminders	• Real-time delivery • Works only for users with the app • Rich notifications (title, body, image)

4 How user preferences determine delivery channels

- Each user has notification preferences stored in a database (e.g. email, SMS, push, or in-app only).
- The worker checks these preferences before sending.
- Users can opt in or out of marketing messages.
- Some notifications (e.g. security alerts) may override preferences to ensure delivery.

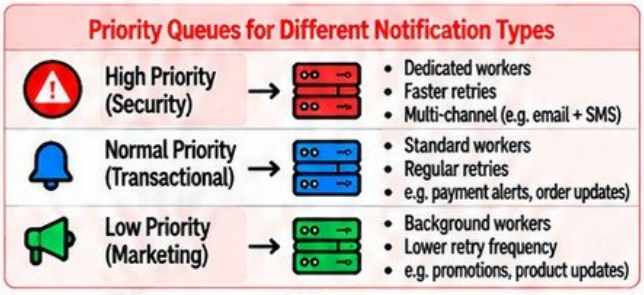
User Notification Preferences (Example)



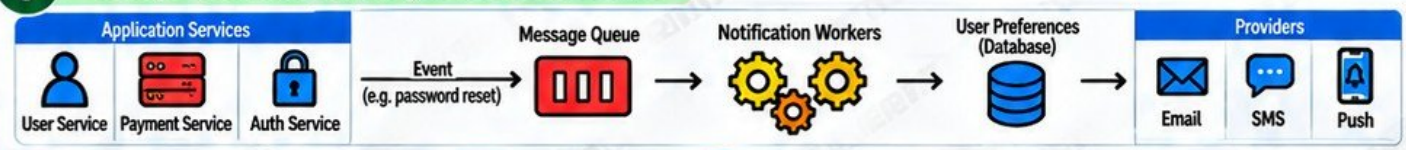
5 Why critical security notifications need different handling

- Security notifications (e.g. password reset, suspicious login) are time-sensitive.
- They should use high-priority queues with dedicated workers.
- They may be sent through multiple channels (email + SMS).
- They require stronger retry logic, monitoring, and alerting.
- They should not be delayed by marketing or non-critical messages.

Priority Queues for Different Notification Types



6 Complete notification system architecture



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

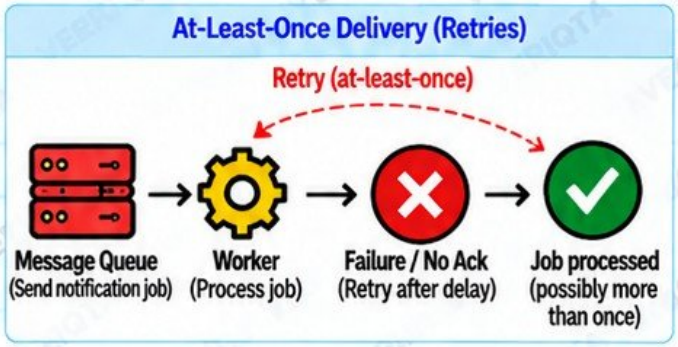
VERIQTA

Why Notifications Fail or Arrive Twice

Scenario: Customers receive duplicate payment alerts while other notifications never arrive.

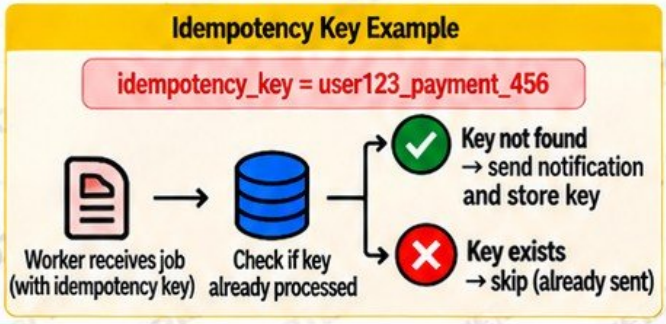
1 How retries and at-least-once delivery work

- Message queues use at-least-once delivery, which means a message may be delivered more than once.
- If a worker fails or does not receive an acknowledgement, the message is retried.
- Retries help prevent message loss but can cause duplicate notifications if processing is not idempotent.
- Network issues, worker crashes, or provider timeouts can trigger retries.



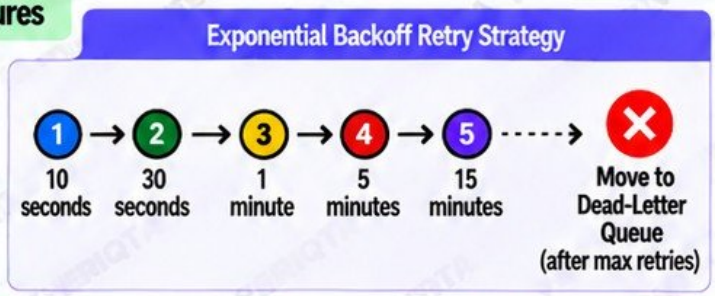
2 How idempotency keys prevent repeated effects

- An idempotency key is a unique identifier for a notification (e.g. user_id + event_type + event_id).
- Before sending, the system checks if this key was already processed.
- If the key exists, the worker skips sending again.
- This prevents duplicate notifications even if the message is retried.



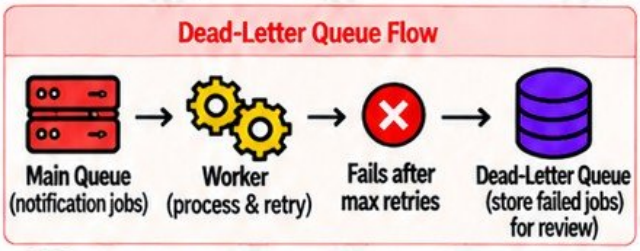
3 How exponential backoff handles provider failures

- If a provider (email, SMS, push) is temporarily down, workers retry with increasing delays.
- Example: 10s → 30s → 1m → 5m → 15m.
- This reduces load on the provider and gives time for recovery.
- After a maximum number of retries, the job is moved to a dead-letter queue.



4 How dead-letter queues isolate failing jobs

- Jobs that keep failing (e.g. invalid email, blocked number, permanent provider error) are moved to a dead-letter queue (DLQ).
- This prevents them from being retried forever.
- Engineers can review DLQ messages, fix the issue, and reprocess if needed.



5 How to distinguish queued, provider-accepted, and actually delivered notifications

Status	What it means	Where to check	Example evidence
Queued	Job is in the queue waiting to be processed	Message queue metrics/logs	Message visible in queue
Processed	Worker processed the job	Worker logs	Log: "Sending notification to provider..."
Provider accepted	Provider received the request	Provider API response (HTTP 200/202)	Response ID from provider (e.g. message_id)
Delivered	Provider confirmed delivery to the user's device/inbox	Provider delivery reports (webhooks or API)	Delivery status: delivered

6 Troubleshooting: Worker reports success, but customer receives nothing

What evidence confirms where delivery stopped?

1. Check queue logs to confirm the job was processed.
2. Review worker logs for the provider response and message ID.
3. Verify the provider dashboard (e.g. SendGrid, Twilio, FCM) for delivery status.
4. Check if a delivery webhook or status callback was received.
5. Inspect user details (correct email/phone/device token).
6. Look for spam filters, blocked numbers, invalid device tokens, or provider-side rejections.

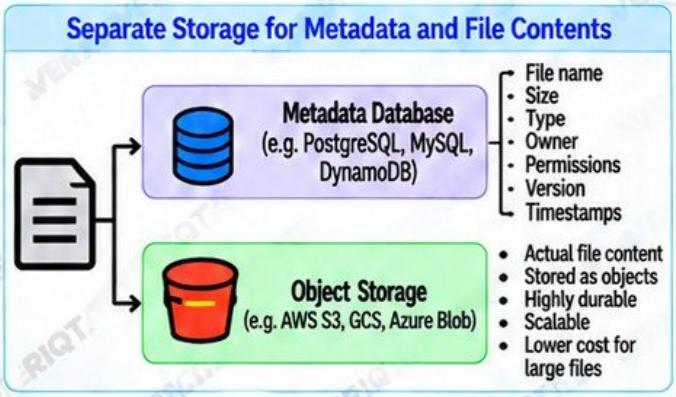
VERIQTA

Design a Cloud File Storage System Like Google Drive

Scenario: Users upload files and access them later from multiple devices.

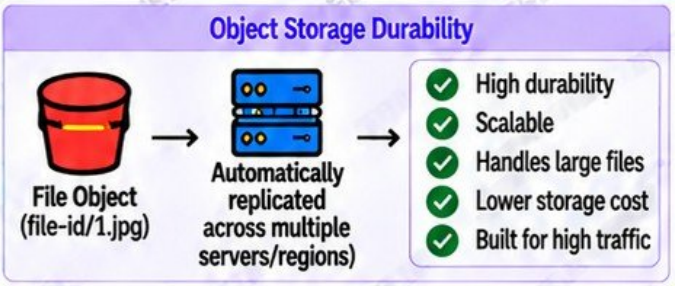
1 Why file metadata and file contents are stored separately

- File metadata (name, size, type, owner, permissions, versions, timestamps) is stored in a database.
- File contents are stored in object storage (e.g. AWS S3, Google Cloud Storage, Azure Blob Storage).
- This separation makes the system scalable, cost-effective, and easier to manage.
- Metadata is small and needs fast search and updates.
- File contents can be very large and require highly durable, low-cost storage.



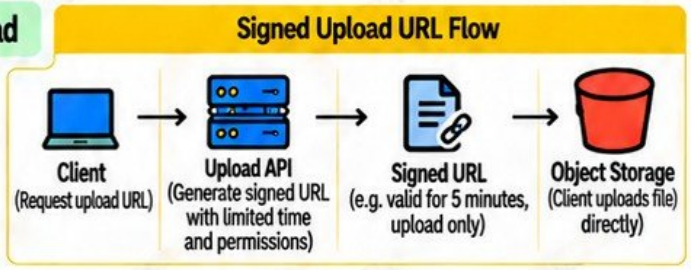
2 How object storage supports durable file storage

- Object storage stores files as objects with a unique key (e.g. file-id/1.jpg).
- It automatically replicates data across multiple servers and locations.
- It provides high durability (e.g. 99.999999999%).
- It can handle very large files and high traffic.



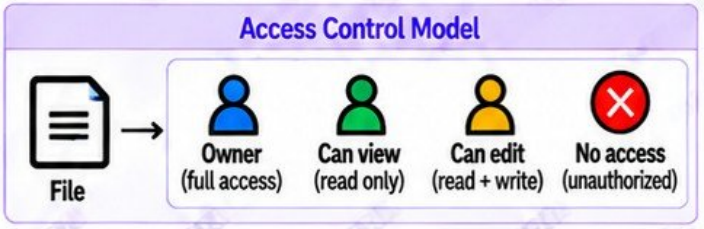
3 How signed upload URLs reduce application-server load

- The client requests a signed upload URL from the application.
- The application generates a time-limited URL with restricted permissions.
- The client uploads the file directly to object storage using this URL.
- This reduces bandwidth and CPU load on the application server.



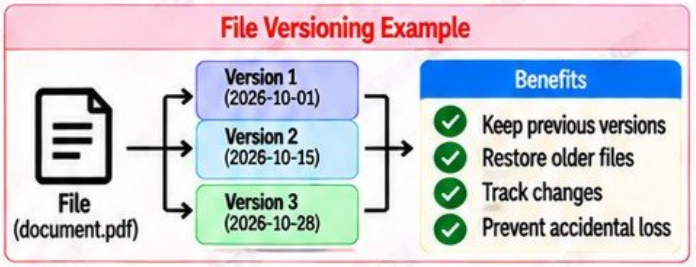
4 How access permissions protect private files

- Each file has an owner and access control list (ACL).
- Permissions define who can view, download, or edit the file.
- Private files are only accessible to authorized users.
- The application checks permissions before generating signed URLs or sharing files.



5 How file versions prevent accidental overwrites

- Each time a file is updated, a new version is created (e.g. file-id/v1, v2, v3).
- Older versions are kept so users can restore previous files.
- Version history helps prevent accidental overwrites and supports recovery from errors.



6 Complete file upload and access flow



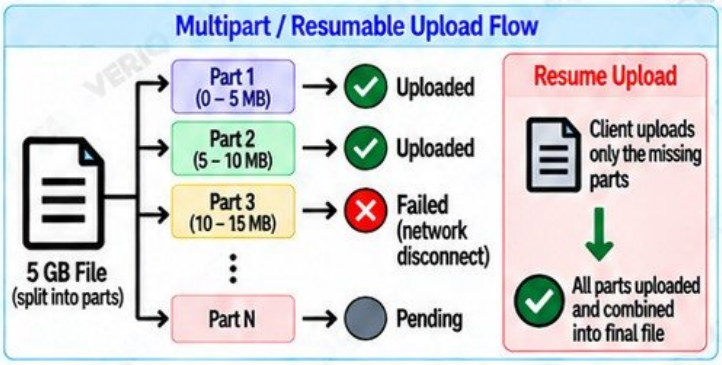
VERIQTA

Handle Interrupted Uploads and File Failures

Scenario: A user uploads a 5 GB file, but the network disconnects at 80%.

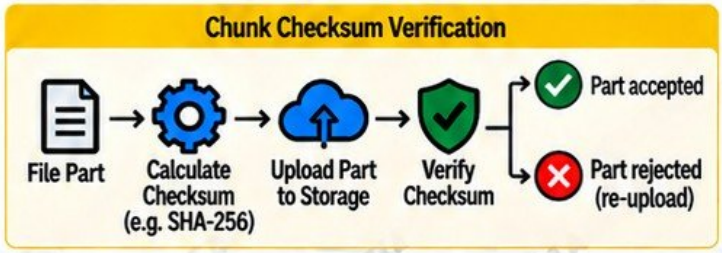
1 How multipart and resumable uploads work

- Large files are split into smaller parts (chunks) e.g. 5 GB file → 5 MB or 10 MB parts.
- Each part is uploaded independently with a unique part ID.
- The service keeps track of uploaded parts.
- If the upload is interrupted, the client resumes by uploading only the missing parts.
- After all parts are uploaded, the service combines the parts into the complete file.



2 Why chunk checksums detect corruption

- Each part has a checksum (e.g. MD5, SHA-256).
- The storage service verifies the checksum when a part is uploaded.
- If the checksum does not match, the part is rejected and re-uploaded.
- This detects network errors or corrupted data before the file is saved.



3 How retrying only failed parts saves bandwidth

- Only the failed or missing parts are retried.
- Successfully uploaded parts are not uploaded again.
- This saves bandwidth, especially for large files.
- The client keeps a record of completed parts and resumes from where it stopped.



4 How abandoned uploads are cleaned up

- If an upload is not completed within a certain time (e.g. 24 hours), it is considered abandoned.
- Incomplete parts are automatically deleted by a lifecycle policy.
- This prevents wasting storage and reduces cost.



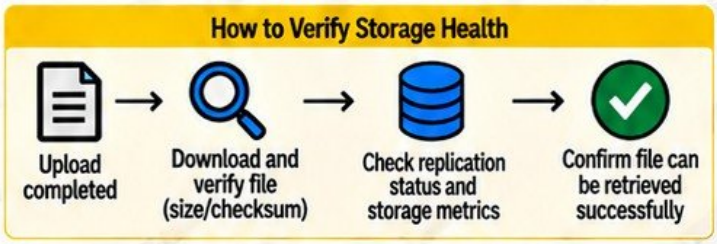
5 How replication, versioning, and recovery protect data

- Files are stored in multiple copies across different servers or regions (replication).
- Versioning keeps previous versions of a file.
- If a file is accidentally deleted or corrupted, an earlier version can be restored.
- These features protect against hardware failures, human errors, and disasters.



6 Production awareness: Verify storage health

- A successful upload response only means the service accepted the file parts.
- Engineers should verify the file can be retrieved later.
- Use automated checks to download and validate files (e.g. size, checksum).
- Monitor storage service metrics, error rates, and replication status.
- Set up alerts for failed uploads, missing parts, or retrieval failures.



VERIQTA

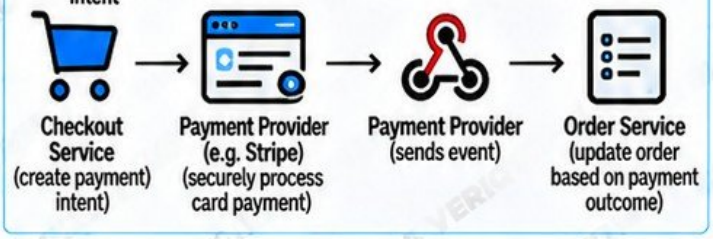
Design an Online Payment System Like Stripe

Scenario: A customer pays for an order, and the store needs a trustworthy payment result.

1 How the checkout service communicates with a payment provider

- The customer enters payment details on the provider's secure payment page or using a provider SDK (e.g. Stripe).
- The checkout service creates a payment intent with the provider (amount, currency, metadata).
- The provider handles card processing, fraud checks, and bank communication.
- The provider returns a client secret or redirect URL for the client to complete the payment.
- The checkout service does not handle raw card details.

High-Level Payment Flow



2 How payment intents and payment states work

- A payment intent represents a single payment attempt.
- It has states such as: requires_payment_method, processing, succeeded, failed, canceled.
- The state changes based on customer action, bank response, or additional verification (e.g. 3D Secure).
- The merchant uses the intent ID to track the payment across the checkout and webhook flow.

Payment Intent States (Example)



3 Why card information should not be handled by the merchant's infrastructure

- Card details are sensitive and fall under security standards like PCI DSS.
- Handling card information increases security risk, compliance cost, and liability.
- A payment provider (e.g. Stripe) securely collects, encrypts, and processes card details.
- The merchant only receives a payment intent ID and payment status, not raw card data.

Secure Payment Collection



4 How payment webhooks report asynchronous outcomes

- Payments can take time to complete (e.g. 3D Secure, bank verification).
- The provider sends webhook events (e.g. payment.succeeded, payment.failed) to the merchant's webhook endpoint.
- The order service updates the order based on these events.
- Webhooks ensure the merchant gets the final, reliable payment result, even if the customer closes the browser.

Webhook Event Flow



5 Why an order and a payment transaction have separate states

- An order represents the customer's intent to buy.
- A payment transaction represents the money movement through the provider and banks.
- An order can be created before the payment succeeds.
- Payment can fail, be pending, or be refunded, while the order state remains separate (e.g. pending payment, paid, canceled).
- Keeping them separate allows better tracking, retries, refunds, and customer support.

Order State vs Payment State

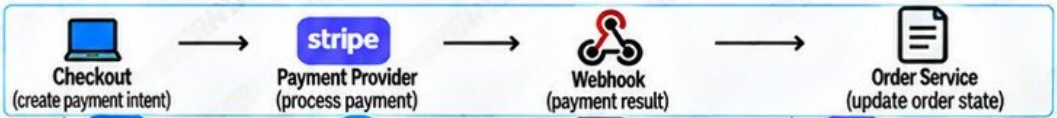
Order States (Example)

- created
- pending_payment
- paid
- shipped
- delivered
- canceled

Payment States (Example)

- requires_payment_method
- processing
- succeeded
- failed
- canceled
- refunded

6 Complete system flow



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

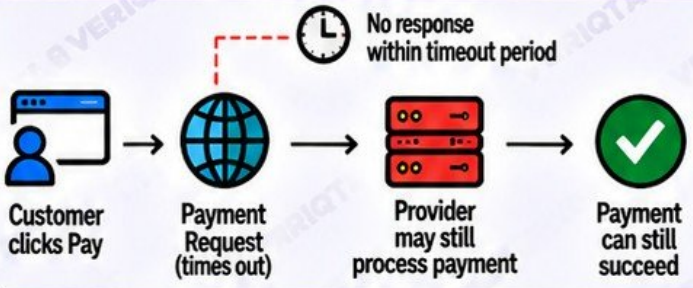
Prevent Double Charges and Lost Payments

Scenario: A customer clicks Pay twice because the first request times out.

1 Why a timeout does not mean a payment failed

- A timeout only means the application did not receive a response in time.
- The payment may have been successfully processed by the provider.
- Network issues, high latency, or provider delays can cause timeouts.
- Always check the payment status with the provider before retrying.
- Retrying blindly can create duplicate charges.

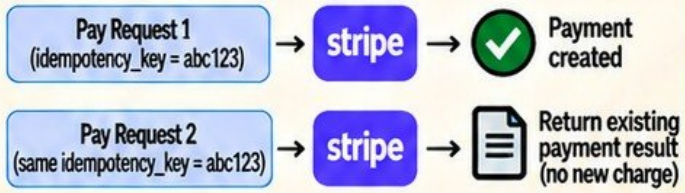
What Happens During a Timeout



2 How idempotency keys prevent duplicate payments

- An idempotency key is a unique identifier for a payment request (e.g. order_id + user_id + timestamp).
- The same key is sent if the request is retried.
- The provider processes only one payment for the key and returns the existing payment result for duplicates.
- This prevents multiple charges even if the customer clicks Pay multiple times or the network retries the request.

Idempotency Key Flow



3 Why webhook signatures must be verified

- Webhooks come from the internet and could be faked by attackers.
- Signature verification confirms the event is genuinely from the provider (e.g. Stripe signature + secret).
- Without verification, attackers could send fake "payment succeeded" events.
- Always verify the signature, timestamp, and event ID before updating orders.

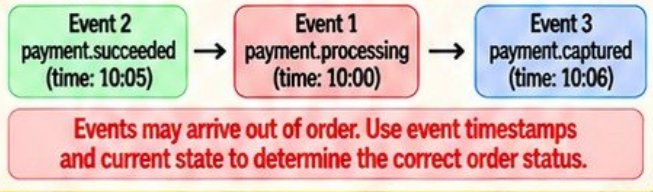
Webhook Verification



4 How delayed and out-of-order events affect order state

- Payment events can arrive late or out of order (e.g. payment.succeeded before payment.processing).
- The system should be able to handle events in any order.
- Use the latest event state and timestamps to update orders correctly.
- Keep a history of events for auditing and debugging.

Out-of-Order Webhook Events (Example)



5 How reconciliation detects disagreements

- Reconciliation compares internal payment records with the provider's records.
- It detects missing payments, duplicate charges, or mismatched statuses.
- Run periodic reconciliation jobs (e.g. daily) using the provider's API to list all payments.
- Investigate and fix any discrepancies.

Reconciliation Flow



6 Engineering decision: Retry or check existing status first?

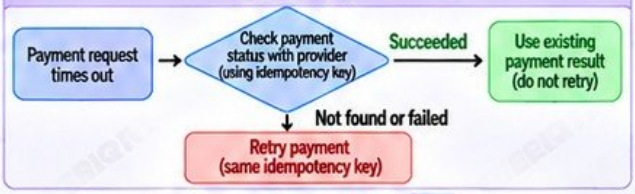
Do NOT retry immediately

- A timeout does not confirm failure.
- Retrying can create duplicate charges without an idempotency key.
- First, retrieve the existing payment status using the payment ID or idempotency key.

Recommended approach

- Use the same idempotency key and check the existing payment status with the provider.
- If the payment succeeded, use that result.
- Retry only if the payment is confirmed as failed or if the provider explicitly allows a retry.

Decision Flow



VERIQTA

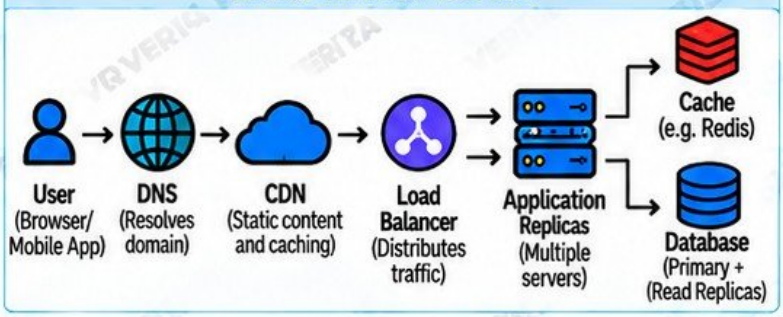
Design a Production Web Application for One Million Users

Scenario: A growing application must serve users reliably across multiple servers.

1 High-level architecture

- Users access the application using a domain name.
- DNS resolves the domain to the nearest CDN or load balancer.
- Static content is served from the CDN.
- Dynamic requests are routed through a load balancer to application replicas.
- Application servers use caches and databases.
- Multiple components work together to handle high traffic and provide reliability.

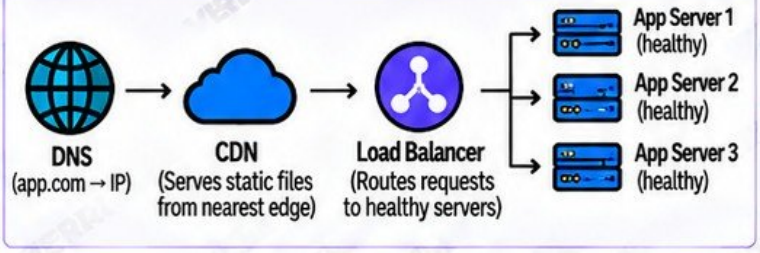
End-to-End Architecture



2 How DNS, CDN, and load balancers work

- DNS converts a domain name (e.g. app.com) to an IP address and often returns the nearest CDN or load balancer.
- A CDN caches static content (images, CSS, JS) at edge locations to reduce latency.
- A load balancer distributes incoming requests across healthy application servers.
- This improves performance, reliability, and global availability.

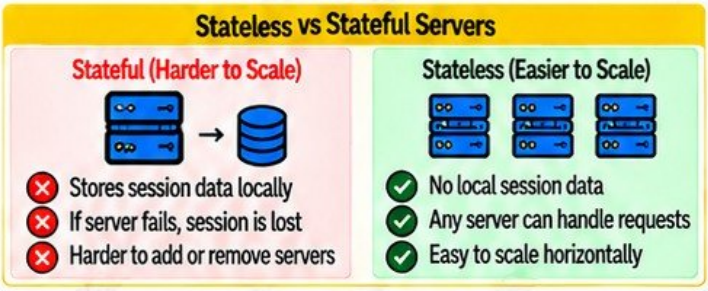
DNS, CDN, and Load Balancer Flow



3 Why stateless application servers are easier to scale

- Stateless servers do not store user session data locally.
- Any server can handle any request.
- This allows new replicas to be added or removed easily.
- Session data is stored in external systems like a database or cache (e.g. Redis).

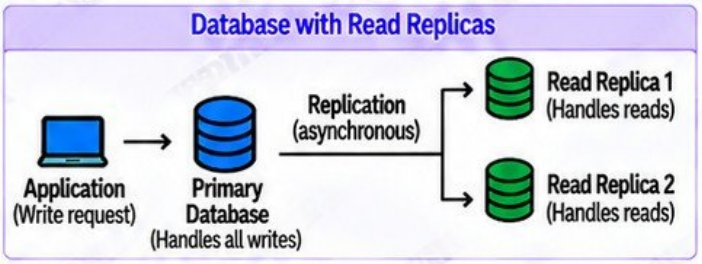
Stateless vs Stateful Servers



4 How database read replicas reduce read workload

- The primary database handles all write operations (e.g. create, update, delete).
- Read replicas handle read operations (e.g. fetch data).
- This reduces load on the primary database.
- Replication keeps the read replicas in sync (with a small delay).

Database with Read Replicas



5 How health checks prevent traffic to unhealthy instances

- Load balancers and Kubernetes perform regular health checks (e.g. HTTP /health endpoint).
- If an instance is unhealthy, it is removed from the list of targets.
- Traffic is only sent to healthy instances.
- This prevents user requests from failing.

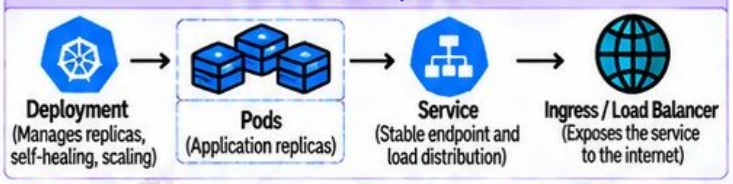
Health Check Flow



6 How Kubernetes Deployments and Services fit in

- A Deployment manages application replicas (pods) and automatically replaces unhealthy pods.
- A Service provides a stable network endpoint for the pods.
- Kubernetes handles scaling, self-healing, and load distribution.
- This makes the application more reliable and easier to operate.

Kubernetes Components



VERIQTA

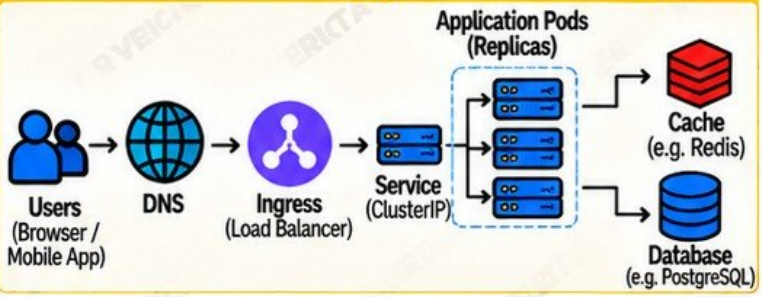
Your Application Is Down. Troubleshoot the Architecture

Scenario: Immediately after a Kubernetes deployment, users report intermittent 502 errors and slow requests.

1 Identify customer impact and recent changes

- Confirm the scope: Are all users affected or only some regions/features?
- Check error rate, latency, and traffic patterns (using monitoring tools).
- Look for recent changes: new deployment, config change, infrastructure update.
- Check if the issue started immediately after the deployment.
- Communicate with stakeholders about the impact.

Architecture Overview (Typical Kubernetes Setup)



2 Inspect Pods, readiness probes, Service endpoints, and ingress routing

- Check pod status, restarts, and readiness.
- Verify readiness and liveness probes.
- Confirm Service endpoints point to healthy pods.
- Check ingress configuration and routing rules.
- Ensure the new deployment pods are receiving traffic and are stable.

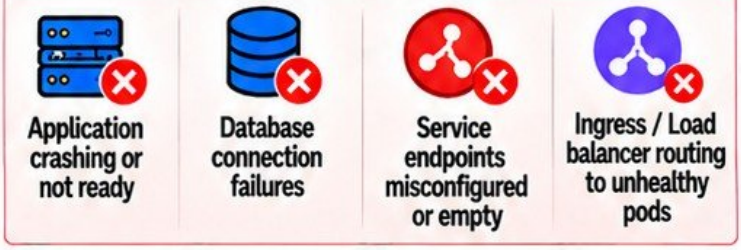
Key Kubernetes Components to Check



3 Distinguish application failures from upstream connection problems

- 502 errors can mean the application is not responding, or an upstream service (e.g. database, API) is failing.
- Check application logs for errors and timeouts.
- Verify database and cache connectivity.
- Test the application directly inside the cluster to isolate the issue.
- Determine if the issue is in the app, a dependency, Ingress, or the load balancer.

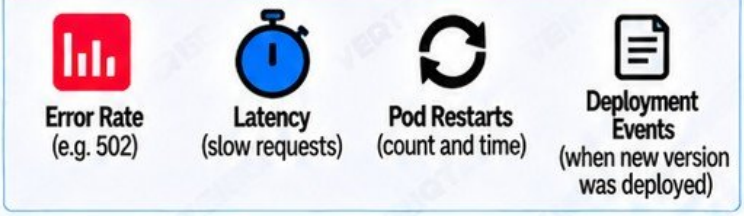
Common 502 Error Causes



4 Correlate metrics and events

- Look at error rates (e.g. 502), request latency, and pod restarts.
- Check deployment events and timestamps.
- Correlate when the new version was deployed with when the issue started.
- Use monitoring and logging to find patterns.

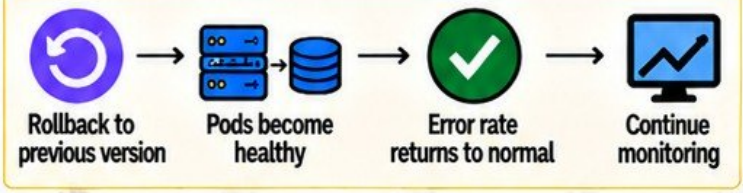
What to Correlate



5 Decide on rollback and verify recovery

- If the new deployment is the likely cause, consider a rollback to the previous stable version.
- Ensure the rollback is safe and does not cause further issues.
- Verify that pods become healthy, error rates drop, and latency returns to normal.
- Monitor the system for a period after recovery to ensure stability.

Rollback and Recovery Flow



6 Practical kubectl commands

```
$ kubectl get pods                          # Check pod status, restarts, readiness
  kubectl describe pod <pod-name>           # Detailed info (events, probes, errors)
$ kubectl logs <pod-name> --previous        # View logs from previous container instance
$ kubectl get endpointslices                # Check Service endpoints
```

Final Exercise

1. Identify the failing layer (app, service, ingress, database, etc).
2. Collect evidence (pod status, logs, events, metrics).
3. Propose a safe mitigation (e.g. rollback, fix config, scale, restart).
4. Verify that customer requests have recovered (check error rate, latency, and pod health).